

Choice of Activation Function in DNN Models

Alex Caramanico, John McMenimon, Parth Rastogi
University of Maryland, College Park
May 2024

Outline

- ❖ Objectives
- ❖ Introduction
 - Background Information
 - Applications
- ❖ Activation Functions
 - Sigmoid and Tanh
 - Rectified Linear Unit (ReLU)
 - Exponential Linear Unit (ELU)
 - Adaptive Piecewise Linear (APL)
 - Comparisons and Performance Analysis
- ❖ Summary
- ❖ References

Objectives

Introduce concepts:

- Neural networks
- Loss functions
- Activation functions

Using *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark* (Dubey, Singh, Chaudhuri):

- Summarize activation function strengths and weaknesses
- Establish weaknesses of traditional AFs
- Compare performance on image classification, translation, and speech recognition



Introduction

Neural Networks - Basics

Neural Networks consist of layers of neurons, i.e. functions, that take in data, act on it, and send the output to the next layer.

The input layer takes in the initial data vector and the output layer produces the final result. The layers in between the input and output layers are known as the hidden layers because they are not visible to the user.

The number of layers, the number of neurons per layer, and the functions used are determined by the engineers designing the neural network architecture. The hidden layers are not “magic” or “unknowns” as may be mistakenly believed.

Neural Network - Structure

Neuron

- Basic unit of computation
- Receives, processes and passes inputs

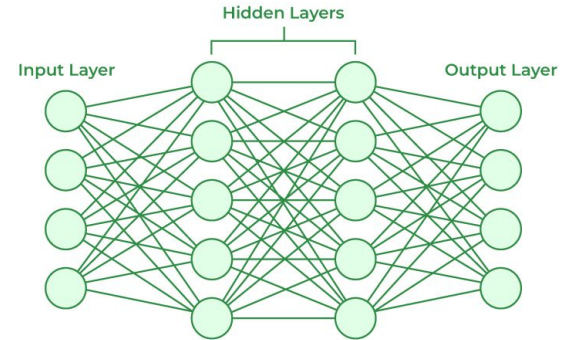
Weights and Biases

- Weight determines the strength and sign of the input
- Biases allow the activation function to be shifted left and right.

$$\hat{y} = \sigma \left(\sum_{i=1}^n x_i w_i + b_i \right)$$

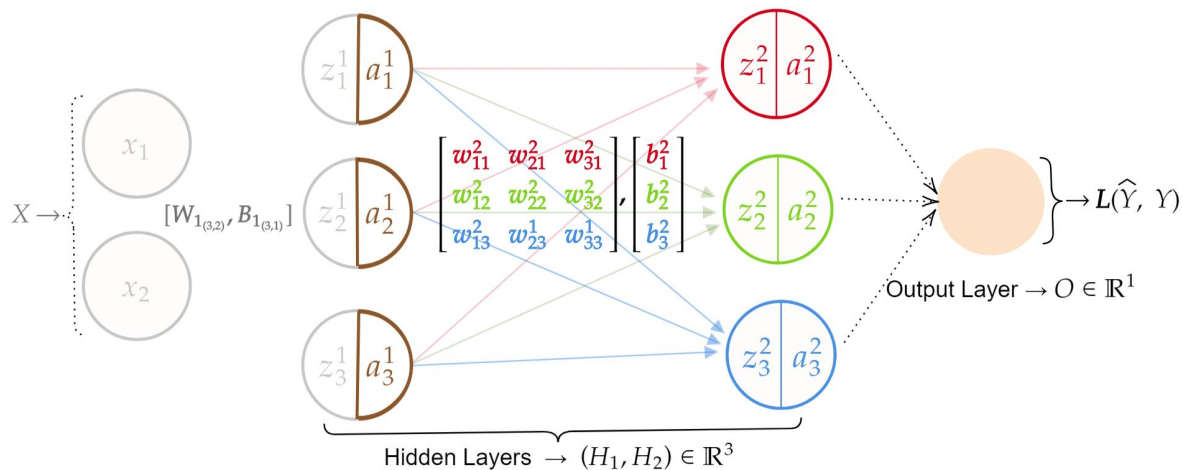
Layers

- Input layer
- Hidden layers
- Output layer



Neural Network - Core

- Forward Propagation
- Backward Propagation
- Activation Function
 - Linear
 - Non-linear
- Loss Function

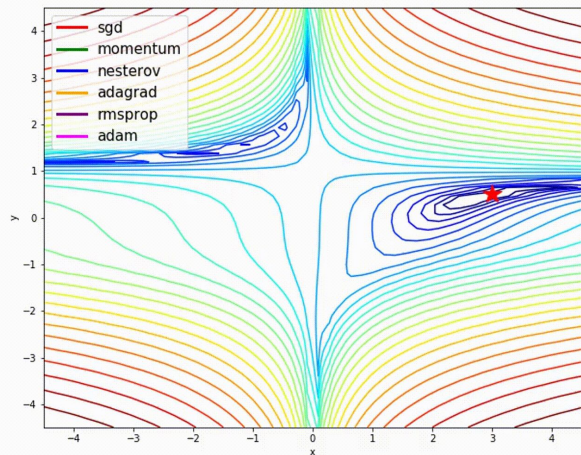


$$\begin{aligned}
 w_{11}^1 x_1 + w_{21}^1 x_2 + b_1^1 &= z_1^1 \\
 w_{12}^1 x_1 + w_{22}^1 x_2 + b_2^1 &= z_2^1 \\
 w_{13}^1 x_1 + w_{23}^1 x_2 + b_3^1 &= z_3^1
 \end{aligned}
 \rightarrow
 \begin{pmatrix} w_{11}^1 & w_{21}^1 \\ w_{12}^1 & w_{22}^1 \\ w_{13}^1 & w_{23}^1 \end{pmatrix}
 \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}
 +
 \begin{pmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{pmatrix}
 =
 \begin{pmatrix} z_1^1 \\ z_2^1 \\ z_3^1 \end{pmatrix}$$

Neural Network - Learning Process

1. Training Data
2. Epochs
3. Batch Size
4. Optimizer

a. ADAM - Adaptive Moment Estimation is an optimization technique for Gradient Decent



Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1]$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

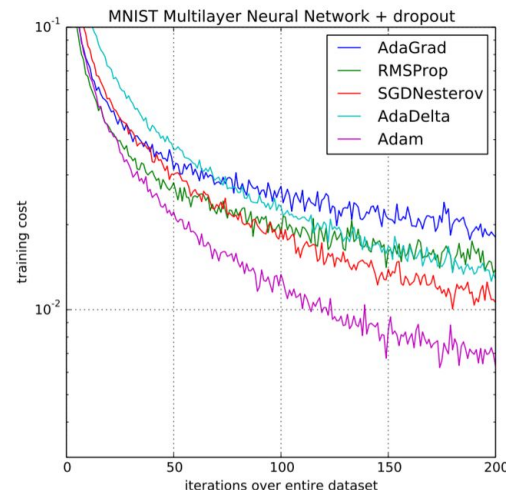
$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)



Neural Networks - Evaluations and Challenges

- Evaluation

- Accuracy
- Precision and Recall
- F1 Score

- Challenges

- Overfitting
- Underfitting
- Vanishing/Exploding Gradients

Potential Applications

Image datasets

- Object detection or identification,
- Image retrieval

Time series

Text datasets - semantic segmentation

Signal datasets

Game datasets

Anomaly detection, skin disease diagnosis, etc.



Common Neural Network Packages



Background - What are activation functions (AFs)?

Activation functions are the functions used by the neurons to act on data input. This presentation will thoroughly explain the major families of activation functions.

Common properties of activation functions are:

- Add nonlinear curvature for the optimization algorithm to descend
- Should not increase computational complexity of the model
- Should not slow down the gradient during training
- Should retain the distribution of data

The tradeoffs between these properties are a key factor in deciding which function to use.

Background - What are activation functions (AFs)?

A neural network should use different activation functions in conjunction.

For example, using a linear activation function in every step of the neural network will return a result equivalent to linear regression. Using a logistic function in every step will return a result equivalent to logistic regression.

Neural networks are more computationally expensive and complex than simple regression models, so there is no point in returning the same results.

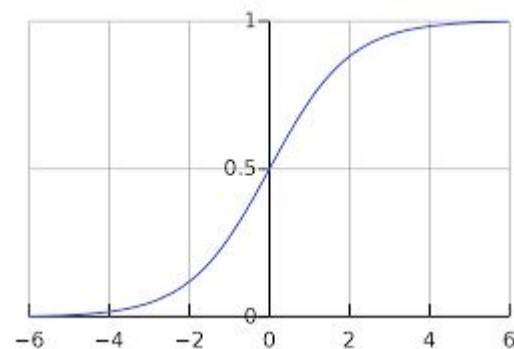
The choice of activation function dramatically influences the performance of the network

Sigmoid and Tanh

Sigmoid and Tanh

Sigmoid:

- Used in models to predict probability as an output [0,1]
- Smooth gradient and clear predictions
- Not zero-centric → poor convergence
- Vanishing gradient
- Not used frequently in practice (but is the go-to pedagogical example)



$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Tanh:

- Classification between two classes [-1, 1]
- Zero-centric
- Vanishing gradient
- Computational complexity

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Varieties: LiSHT, Elliott, SRS

$$LiSHT(x) = x \times Tanh(x) \quad Elliott(x) = \frac{0.5 \times x}{1 + |x|} + 0.5$$

$$SRS(x) = \frac{x}{\frac{x}{\alpha} + e^{-x/\beta}}$$

Table 2: Summary of Logistic Sigmoid and Tanh based activation functions.

Name of AF	Parametric	Monotonic	Smooth	Bounded
Logistic Sigmoid	No	Yes	Yes	Yes
Tanh	No	Yes	Yes	Yes
Scaled Tanh (sTanh), 1998 [18]	Yes	Yes	Yes	Yes
Rectified Hyperbolic Secant (ReSech), 2016 [19]	No	No	Yes	Yes
Scaled Sigmoid (sSigmoid), 2016 [20]	No	Yes	Yes	Yes
Penalized Tanh (pTanh), 2016 [20]	No	Yes	No	Yes
Hexpo, 2017 [21]	No	Yes	Yes	Yes
Improved Sigmoid (ISigmoid), 2018 [22]	No	Yes	Yes	No
Sigmoid-Weighted Linear Units (SiLU), 2018 [23]	No	No	Yes	For negative inputs
Linearly Scaled Hyperbolic Tangent (LiSHT), 2019 [24]	No	No	Yes	No
Elliott, 2019 [25]	No	Yes	Yes	Yes
Soft-Root-Sign (SRS), 2020 [26]	Yes	No	Yes	Yes

From Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark. Dubey, Singh, Chaudhuri

Rectified Linear Unit (ReLU)

Rectified Linear Unit (ReLU)

$$f(x) = \max(0, x)$$

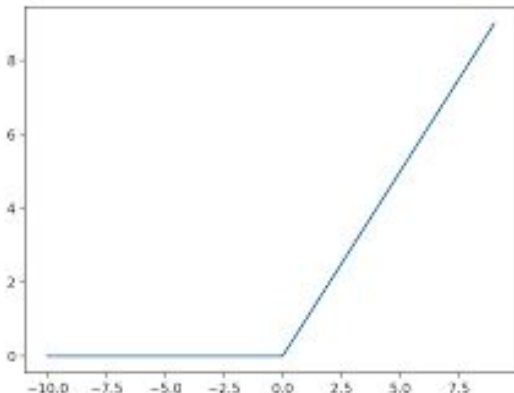
Used in convolutional neural networks

- Filters optimization
- Self-learning feature engineering

Image classification, semantics

Under-usage of negative values → poor convergence

Manual design



Leaky ReLU (LReLU):

- Solved dead ReLU problem: derivative exists at negative inputs $f(x) = \max(0.01x, x)$

Others: PReLU (Parametric), ABRReLU (Average Biased)

$$PReLU(x) = \begin{cases} x, & x \geq 0 \\ p \times x, & x < 0 \end{cases} \quad ABRReLU(x) = \begin{cases} x - \beta, & x - \beta \geq 0 \\ 0, & x - \beta < 0 \end{cases}$$

Table 3: Summary of Rectified Linear Unit based activation functions.

Name	Parametric	Monotonic	Smooth	Bounded
Rectified Linear Unit (ReLU), 2010 [17]	No	Yes	No	For negative inputs
Leaky ReLU (LReLU), 2013 [34]	No	Yes	No	No
Parametric ReLU (PReLU), 2015 [35]	Yes	Yes	No	No
Randomized ReLU (RReLU), 2015 [35]	No	Yes	No	No
Concatenated ReLU (CReLU), 2016 [36]	No	Yes	No	For negative inputs
Bounded ReLU (BReLU), 2016 [37]	No	Yes	No	Yes
Parametric Tanh Linear Unit (PTeLU), 2017 [38]	Yes	Yes	Yes	For negative inputs
Flexible ReLU (FReLU), 2018 [39]	Yes	Yes	No	For negative inputs
Elastic ReLU (EReLU), 2018 [40]	No	Yes	No	For negative inputs
Randomly Translational ReLU (RTRReLU), 2018 [41]	No	Yes	No	For negative inputs
Dual ReLU (DualReLU), 2018 [42]	No	Yes	No	No
Paired ReLU (PairedReLU), 2018 [43]	Yes	Yes	No	No
Average Biased ReLU (ABReLU), 2018 [44]	No	Yes	No	For negative inputs
Natural-Logarithm (NLReLU), 2019 [45]	No	Yes	No	For negative inputs
Multi-bin Trainable Linear Units (MTLU), 2019 [46]	Yes	No	No	No
Lipschitz ReLU (L-ReLU), 2020 [47]	Yes	Depends upon ϕ and η	Depends upon ϕ and η	Depends upon ϕ and η

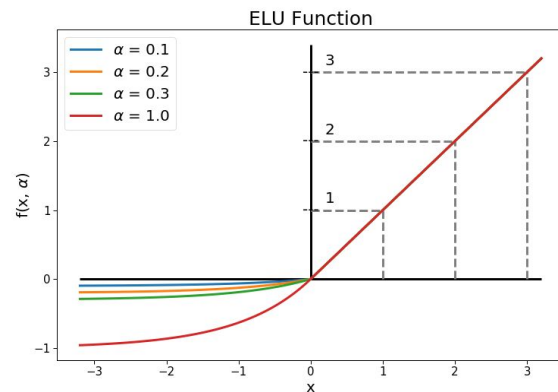
From Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark. Dubey, Singh, Chaudhuri

Exponential Linear Unit (ELU)

Exponential Linear Unit (ELU)

$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha(e^x - 1), & \text{otherwise} \end{cases}$$

- Parameter alpha can be learned from back propagation
- No dead ReLU issues
- Mean close to zero-centered
- Differentiable, saturated for negative values, reduces bias shift
- Manual Design
- Relatively computationally expensive
- Varieties:
 - Scaled (SELU) - whole thing scaled by λ
 - Continuously Differentiable (CELU)
 - Parametric Deformable (PDELU)



$$CELU(x) = \begin{cases} x, & x \geq 0 \\ \alpha \times (e^{x/\alpha} - 1), & x < 0 \end{cases} \quad PDELU(x) = \begin{cases} x, & x > 0 \\ \alpha \times ([1 + (1 - t) \times x]^{\frac{1}{1-t}} - 1), & x \leq 0 \end{cases}$$

Table 4: Summary of Exponential Linear Unit based activation functions.

Name	Parametric	Monotonic	Smooth	Bounded
Exponential Linear Unit (ELU), 2016 [27]	Yes	Yes	Yes	For negative inputs
Scaled ELU (SELU), 2017 [52]	Yes	Yes	Yes	For negative inputs
Continuously Differentiable ELU (CELU), 2017 [53]	Yes	Yes	No	For negative inputs
Parametric ELU (PELU), 2017 [54]	Yes	Yes	No	For negative inputs
Multiple PELU (MPELU), 2018 [55]	Yes	Yes	No	For negative inputs
Fast ELU (FELU), 2019 [56]	Yes	Yes	No	For negative inputs
Parametric Rectified Exponential Unit (PREU), 2019 [57]	Yes	No	Yes	For negative inputs
Elastic ELU (EELU), 2020 [58]	Yes	Yes	No	For negative inputs
Parametric Deformable ELU (PDELU), 2020 [59]	Yes	Yes	Yes	For negative inputs

From *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. Dubey, Singh, Chaudhuri

Adaptive Piecewise Linear (APL)

Adaptive Piecewise Linear (APL) $APL(x) = \max(0, x) + \sum_{s=1}^S a_s \times \max(0, b_s - x)$

- Recent trend, adjust based on dataset complexity
- Learnable parameters
 - a and b , hyperparameter S = number of hinges
- Output range 0 to infinity
- Swish
 - Automatic search
 - Shape adjusted between linear and ReLU
 - Controls amount of non-linearity
$$Swish(x) = x \times Sigmoid(\beta \times x)$$

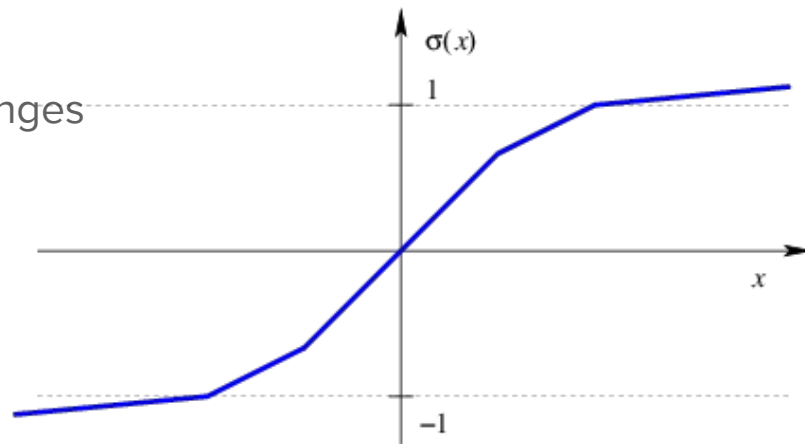


Table 5: Summary of adaptive and learning based activation functions.

Name	Parametric	Monotonic	Smooth	Bounded
Adaptive Piecewise Linear Unit (APL), 2015 [28]	Yes	No	No	No
Spline AF (SAF), 2016 [74]	Yes	Yes	Yes	No
Bi-Modal Derivative Adaptive Activation (BDAA), 2017 [75]	Yes	Yes	Yes	Yes
Adaptive AF (AAF), 2018 [76]	Yes	Yes	No	No
Swish, 2018 [29]	Yes	No	Yes	No
ESwish, 2018 [77]	Yes	No	Yes	No
Trainable AF (TAF), 2018 [78]	Yes	No	Yes	No
Self-Learnable AF (SLAF), 2019 [79]	Yes	No	Yes	No
Mexican ReLU (MeLU), 2019 [80]	Yes	No	No	No

From Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark. Dubey, Singh, Chaudhuri

Table 1: Advantage and disadvantage of primary AFs.

AFs	Diminishing gradients	Limited non-linearity	Optimization difficulty	Lack of adaptability	Computational inefficiency
Sigmoid	Yes	No	Yes	Yes	Yes
Tanh	Yes	No	Partial	Yes	Yes
ReLU	Partial	Yes	Partial	Yes	No
ELU	No	Partial	No	Yes	Partial
APL	No	Partial	No	No	No
Swish	No	Partial	No	No	Partial

From *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. Dubey, Singh, Chaudhuri

Other Functions

- Softplus

- Statistical applications
- Smooth, facilitating differentiability
- Trainable parameters
- Mish: non-monotonic, smooth

$$SLU(x) = \begin{cases} \alpha \times x, & x \geq 0 \\ \beta \times \log(e^x + 1) - \gamma, & x < 0 \end{cases}$$

$$Mish(x) = x \times \tanh(\text{Softplus}(x))$$

- Probabilistic (stochastic) functions

- Rare - expensive sampling process
- GELU - Gaussian Error Linear Unit

$$GELU(x) = x \times P(X \leq x)$$

- Polynomial functions

- Light-weight models, suitable for FPGAs and microcontrollers
- Pade Activation Unit (PAU)
 - $P(x)$ and $Q(x)$ polynomials of order m and n respectively

$$PAU(x) = \frac{P(x)}{Q(x)}$$

- Kernel functions

- Complex-valued neural networks

AF Comparisons - Experimental Results

On Image Classification and Translation / Speech Recognition datasets

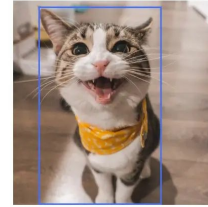
18 Activation Functions tested over 6 Convolutional Neural Networks (CNNs)

Image Classification

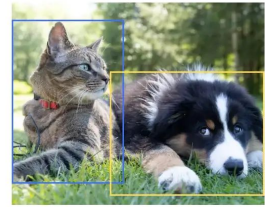
- 50,000 training images, 10,000 test images
 - CIFAR10 - 10 object categories
 - CIFAR100 - 100 object categories
- Training over 100 Epochs
- Constant batch size of 128
- Learning rate 0.001 for first 80 Epochs, 0.0001 for last 20 Epochs
- Data normalization performed
- Adam optimizer utilized
- Accuracy metric - percentage of correct classifications
 - 5 trials run for each activation function
 - Mean scores and standard deviations reported



Classification
Cat



Classification, Localization
Cat



Object Detection
Cat, Dog

Table 8: Experimental results comparison over CIFAR10 dataset.

Accuracy Activations	CNN Models					
	MobileNet	VGG16	GoogleNet	ResNet50	SENet18	DenseNet121
Sigmoid	88.60 $\pm$ 0.17	87.69 $\pm$ 0.49	87.33 $\pm$ 2.48	80.13 $\pm$ 3.33	90.29 $\pm$ 0.29	89.92 $\pm$ 1.96
Tanh	87.21 $\pm$ 0.24	90.49 $\pm$ 0.11	90.16 $\pm$ 1.86	89.09 $\pm$ 1.47	90.44 $\pm$ 0.09	91.80 $\pm$ 0.69
Elliott [25]	88.48 $\pm$ 0.18	87.94 $\pm$ 0.49	89.84 $\pm$ 3.43	81.60 $\pm$ 3.91	90.25 $\pm$ 0.25	91.53 $\pm$ 1.04
ReLU [8]	90.10 $\pm$ 0.22	92.84 $\pm$ 0.19	93.43 $\pm$ 0.48	93.74 $\pm$ 0.34	93.70 $\pm$ 0.16	93.96 $\pm$ 0.51
LReLU [17]	90.10 $\pm$ 0.19	91.09 $\pm$ 0.09	89.28 $\pm$ 0.82	93.83 $\pm$ 0.42	93.66 $\pm$ 0.19	93.85 $\pm$ 0.48
PReLU [35]	90.43 $\pm$ 0.18	92.19 $\pm$ 0.08	92.85 $\pm$ 0.55	92.99 $\pm$ 0.62	92.76 $\pm$ 0.26	92.82 $\pm$ 0.63
ELU [27]	90.92 $\pm$ 0.25	88.55 $\pm$ 1.17	92.47 $\pm$ 0.76	93.53 $\pm$ 0.66	93.39 $\pm$ 0.20	92.89 $\pm$ 0.62
SELU [52]	90.11 $\pm$ 0.32	92.25 $\pm$ 0.28	91.87 $\pm$ 0.84	93.53 $\pm$ 0.52	89.96 $\pm$ 0.31	92.71 $\pm$ 0.73
GELU [101]	90.71 $\pm$ 0.20	92.42 $\pm$ 0.09	93.16 $\pm$ 0.61	93.81 $\pm$ 0.46	93.72 $\pm$ 0.18	93.90 $\pm$ 0.41
CELU [53]	91.04 $\pm$ 0.17	88.11 $\pm$ 0.14	92.60 $\pm$ 0.60	94.09 $\pm$ 0.17	91.63 $\pm$ 0.22	93.46 $\pm$ 0.35
Softplus [93]	91.05 $\pm$ 0.22	92.69 $\pm$ 0.20	92.66 $\pm$ 0.66	93.34 $\pm$ 0.65	93.25 $\pm$ 0.11	93.07 $\pm$ 0.70
Swish [29]	90.66 $\pm$ 0.34	92.32 $\pm$ 0.20	92.68 $\pm$ 0.53	93.02 $\pm$ 0.85	93.24 $\pm$ 0.19	93.16 $\pm$ 0.51
ABReLU [44]	88.97 $\pm$ 0.47	92.36 $\pm$ 0.15	93.34 $\pm$ 0.23	93.29 $\pm$ 0.52	93.35 $\pm$ 0.14	93.26 $\pm$ 0.55
LiSHT [24]	86.53 $\pm$ 0.49	89.83 $\pm$ 0.28	90.27 $\pm$ 0.80	90.89 $\pm$ 0.66	90.25 $\pm$ 0.84	87.91 $\pm$ 0.93
SRS [26]	89.43 $\pm$ 0.81	92.06 $\pm$ 0.26	91.36 $\pm$ 1.19	92.28 $\pm$ 0.48	78.05 $\pm$ 1.37	90.64 $\pm$ 1.93
Mish [99]	90.82 $\pm$ 0.15	92.85 $\pm$ 0.25	93.29 $\pm$ 0.61	93.69 $\pm$ 0.63	93.66 $\pm$ 0.12	93.62 $\pm$ 0.62
PAU [111]	90.67 $\pm$ 0.17	92.00 $\pm$ 0.26	92.80 $\pm$ 0.65	93.67 $\pm$ 0.52	93.08 $\pm$ 0.20	93.05 $\pm$ 0.53
PDELU [59]	90.18 $\pm$ 0.19	92.80 $\pm$ 0.13	93.49 $\pm$ 0.30	93.42 $\pm$ 0.71	93.71 $\pm$ 0.07	93.96 $\pm$ 0.59

From *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. Dubey, Singh, Chaudhuri

Table 9: Experimental results comparison over CIFAR100 dataset.

Accuracy Activations	CNN Models					
	MobileNet	VGG16	GoogleNet	ResNet50	SENet18	DenseNet121
Sigmoid	61.88 $\pm$ 0.18	37.75 $\pm$ 0.59	70.31 $\pm$ 0.54	46.78 $\pm$ 5.42	66.17 $\pm$ 1.16	68.31 $\pm$ 2.41
Tanh	53.10 $\pm$ 0.51	58.43 $\pm$ 0.38	67.66 $\pm$ 2.32	64.32 $\pm$ 1.69	60.13 $\pm$ 1.86	69.53 $\pm$ 1.68
Elliott [25]	60.70 $\pm$ 0.34	33.20 $\pm$ 0.97	64.85 $\pm$ 6.28	49.88 $\pm$ 4.03	66.30 $\pm$ 0.28	69.58 $\pm$ 2.40
ReLU [8]	61.33 $\pm$ 0.34	67.47 $\pm$ 0.44	74.05 $\pm$ 1.69	71.96 $\pm$ 0.94	70.45 $\pm$ 0.73	72.99 $\pm$ 1.35
LReLU [17]	61.13 $\pm$ 0.41	65.72 $\pm$ 0.14	63.79 $\pm$ 2.38	72.77 $\pm$ 0.49	70.58 $\pm$ 0.45	73.33 $\pm$ 1.25
PReLU [35]	59.86 $\pm$ 0.35	65.26 $\pm$ 0.40	69.57 $\pm$ 1.50	71.08 $\pm$ 1.70	69.77 $\pm$ 0.48	68.23 $\pm$ 1.55
ELU [27]	61.97 $\pm$ 0.24	51.35 $\pm$ 3.01	72.57 $\pm$ 1.76	71.41 $\pm$ 1.63	71.27 $\pm$ 0.58	72.06 $\pm$ 1.93
SELU [52]	59.62 $\pm$ 0.39	64.55 $\pm$ 0.43	71.47 $\pm$ 1.39	69.94 $\pm$ 1.92	55.01 $\pm$ 0.98	70.15 $\pm$ 1.04
GELU [101]	61.20 $\pm$ 0.61	67.25 $\pm$ 0.38	74.27 $\pm$ 0.70	71.58 $\pm$ 0.87	71.14 $\pm$ 0.29	73.31 $\pm$ 1.70
CELU [53]	61.90 $\pm$ 0.21	55.78 $\pm$ 0.69	72.87 $\pm$ 1.52	70.95 $\pm$ 1.40	63.43 $\pm$ 0.81	72.68 $\pm$ 1.16
Softplus [93]	62.59 $\pm$ 0.21	67.70 $\pm$ 0.19	73.08 $\pm$ 1.66	71.99 $\pm$ 2.03	71.16 $\pm$ 0.46	72.54 $\pm$ 1.73
Swish [29]	59.40 $\pm$ 0.41	66.05 $\pm$ 0.82	71.56 $\pm$ 1.66	71.12 $\pm$ 2.08	68.42 $\pm$ 1.62	71.34 $\pm$ 1.10
ABReLU [44]	56.21 $\pm$ 0.53	66.95 $\pm$ 0.09	71.83 $\pm$ 2.26	71.96 $\pm$ 1.43	70.47 $\pm$ 0.91	73.79 $\pm$ 1.45
LiSHT [24]	54.09 $\pm$ 1.54	58.87 $\pm$ 0.81	66.66 $\pm$ 2.50	65.28 $\pm$ 1.33	66.01 $\pm$ 1.04	65.61 $\pm$ 1.10
SRS [26]	54.93 $\pm$ 0.80	58.22 $\pm$ 1.09	70.39 $\pm$ 1.09	67.11 $\pm$ 1.46	36.95 $\pm$ 0.93	64.52 $\pm$ 1.39
Mish [99]	61.81 $\pm$ 0.54	68.13 $\pm$ 0.40	73.76 $\pm$ 1.48	71.89 $\pm$ 1.12	70.80 $\pm$ 0.68	73.49 $\pm$ 1.39
PAU [111]	59.81 $\pm$ 0.61	64.14 $\pm$ 0.62	70.48 $\pm$ 1.53	68.59 $\pm$ 2.15	68.29 $\pm$ 0.77	67.83 $\pm$ 0.35
PDELU [59]	61.35 $\pm$ 0.56	67.92 $\pm$ 0.32	74.48 $\pm$ 1.23	72.11 $\pm$ 1.60	70.81 $\pm$ 0.47	73.71 $\pm$ 1.64

From *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. Dubey, Singh, Chaudhuri

Table 10: Training time (hh:mm:ss) comparison over CIFAR100 dataset.

Training Time Activations	CNN Models					
	MobileNet	VGG16	GoogleNet	ResNet50	SENet18	DenseNet121
Sigmoid	00:33:15	00:49:16	04:55:54	03:36:03	01:13:14	04:12:24
Tanh	00:33:18	00:49:55	04:58:02	03:33:03	01:13:18	04:09:24
Elliott [25]	00:49:52	00:59:13	06:53:55	05:38:49	01:41:38	07:46:55
ReLU [8]	00:31:22	00:47:19	04:55:10	03:32:30	01:15:33	04:15:06
LReLU [34]	00:31:48	00:49:03	05:01:30	03:33:00	01:18:38	04:14:09
PReLU [35]	00:44:24	00:49:01	05:42:18	03:55:57	01:27:05	04:55:47
ELU [27]	00:31:05	00:47:38	04:57:37	03:36:47	01:13:25	04:08:39
SELU [52]	00:29:40	00:47:31	04:54:57	03:33:47	01:13:27	04:09:17
GELU [101]	00:29:43	00:47:22	04:55:53	03:32:32	01:13:32	04:11:26
CELU [53]	00:29:36	00:46:47	05:00:44	03:31:40	01:14:08	04:18:11
Softplus [93]	00:29:44	00:47:06	04:58:55	03:32:03	01:14:02	04:12:08
Swish [29]	00:43:13	00:55:37	06:18:38	04:58:38	01:32:15	06:41:14
ABReLU [44]	00:38:51	00:53:49	05:43:59	04:27:02	01:25:30	05:42:53
LiSHT [24]	00:37:01	00:54:10	05:40:00	04:25:57	01:23:59	05:38:15
SRS [26]	01:06:38	01:11:36	08:43:09	07:35:35	02:05:33	11:10:27
Mish [99]	00:40:19	00:54:23	05:59:48	04:46:45	01:28:53	06:10:27
PAU [111]	00:41:59	00:54:10	05:54:22	04:12:31	01:25:37	05:39:57
PDELU [59]	05:23:38	04:01:55	34:22:00	36:48:48	08:32:40	50:23:00

From *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. Dubey, Singh, Chaudhuri

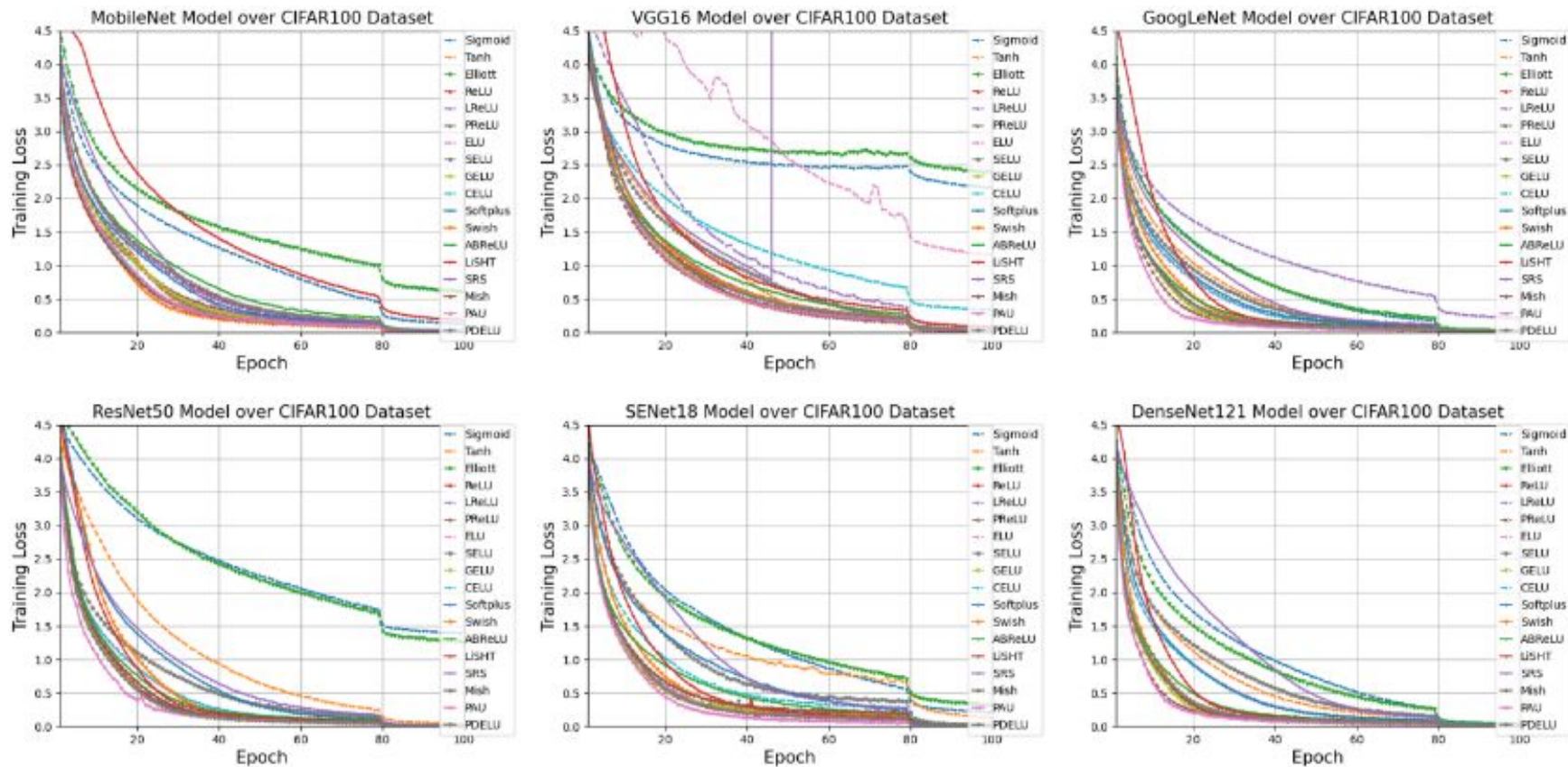
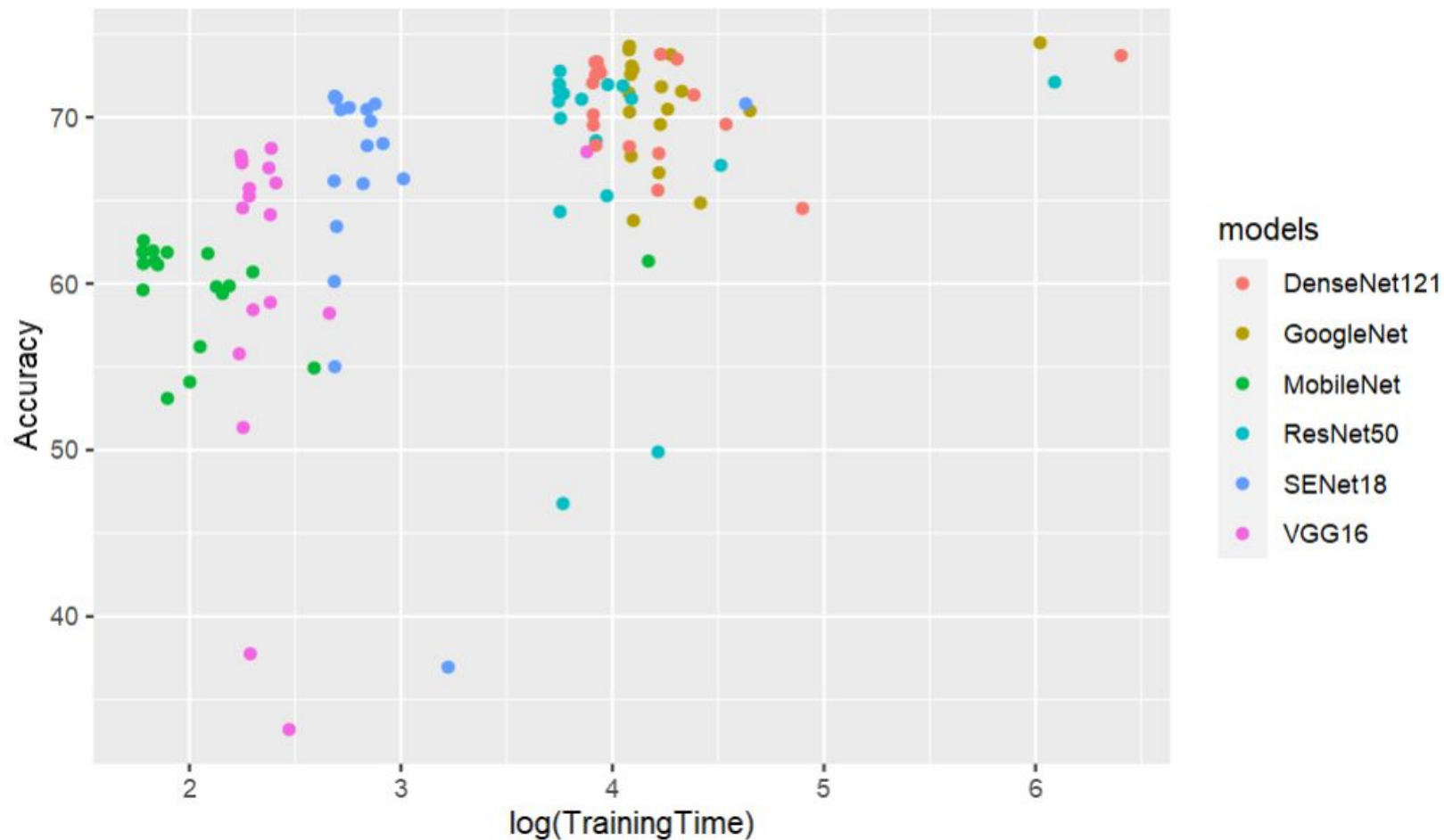


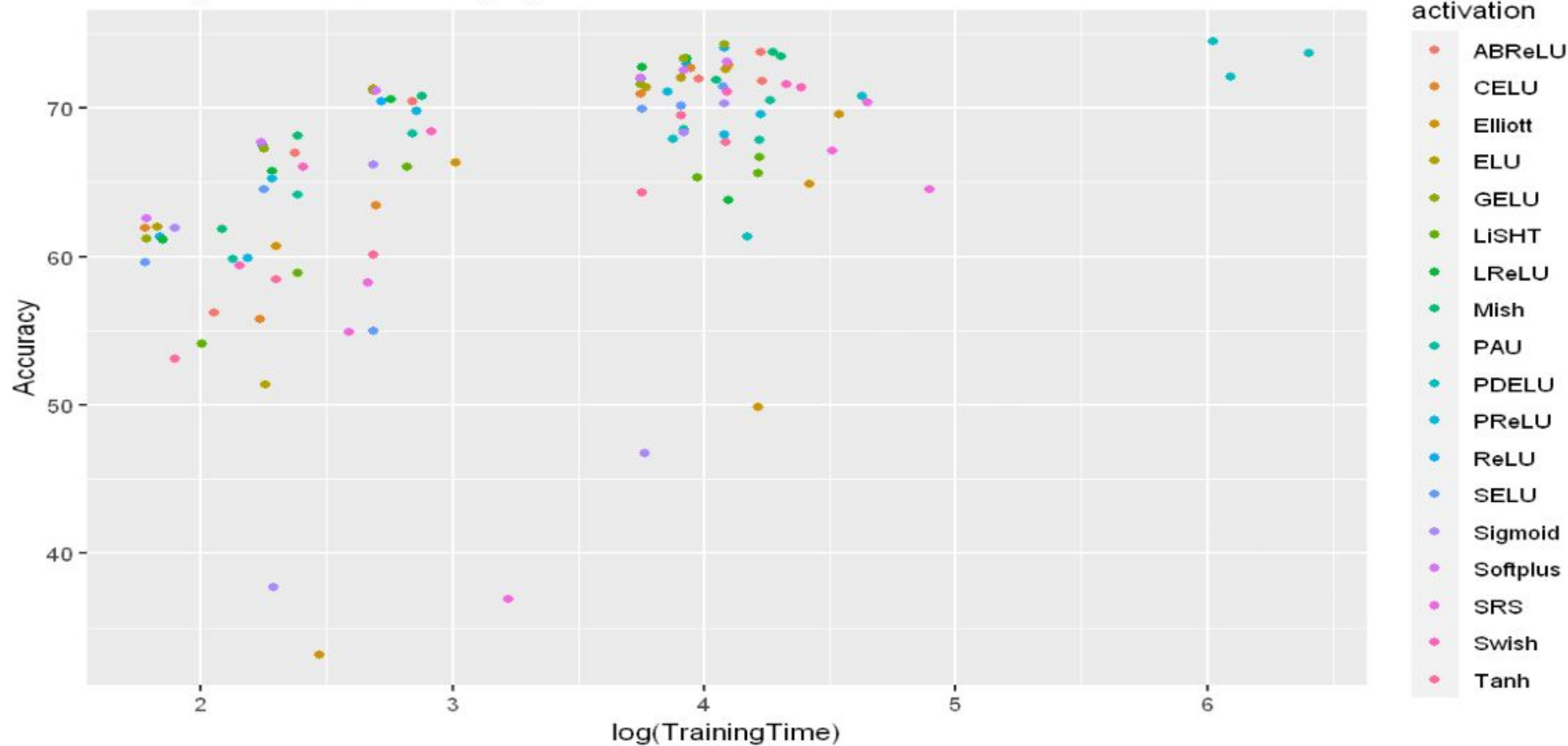
Figure 3: Convergence plots over CIFAR100 dataset.

From *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. Dubey, Singh, Chaudhuri

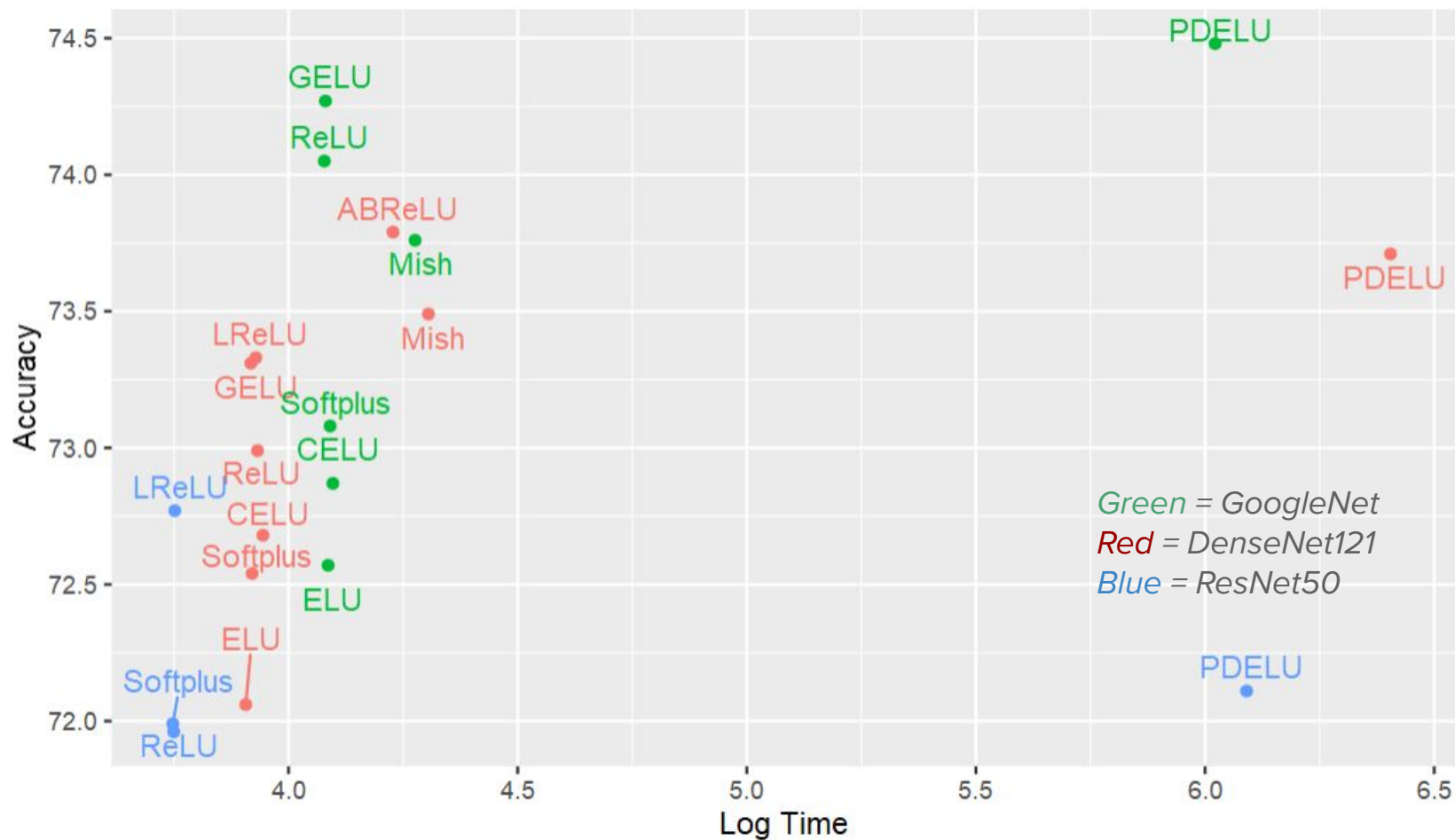
Training Time vs Accuracy by Model



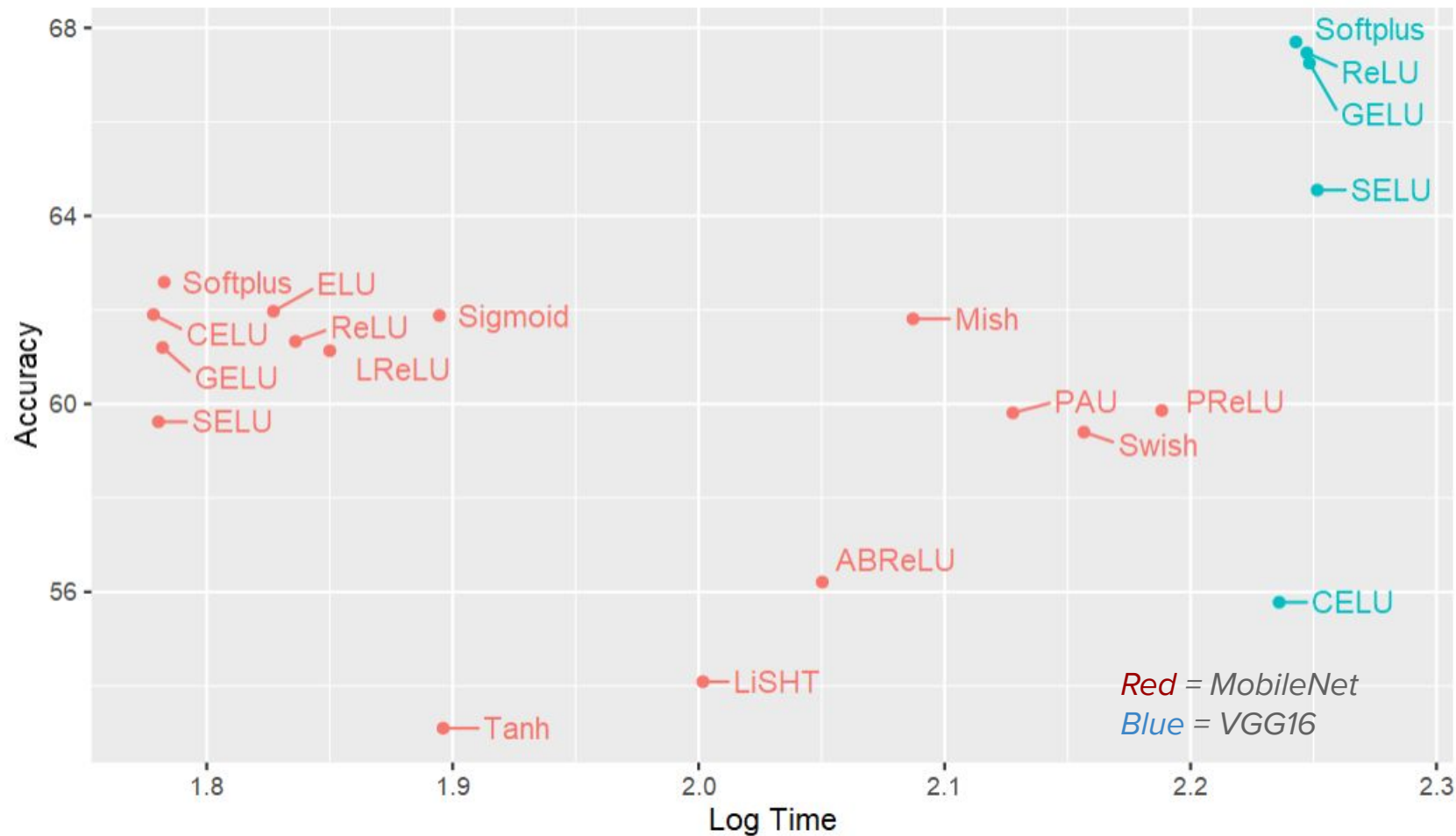
Training Time vs Accuracy by AF



Top 20 Most Accurate Model-AF Performances

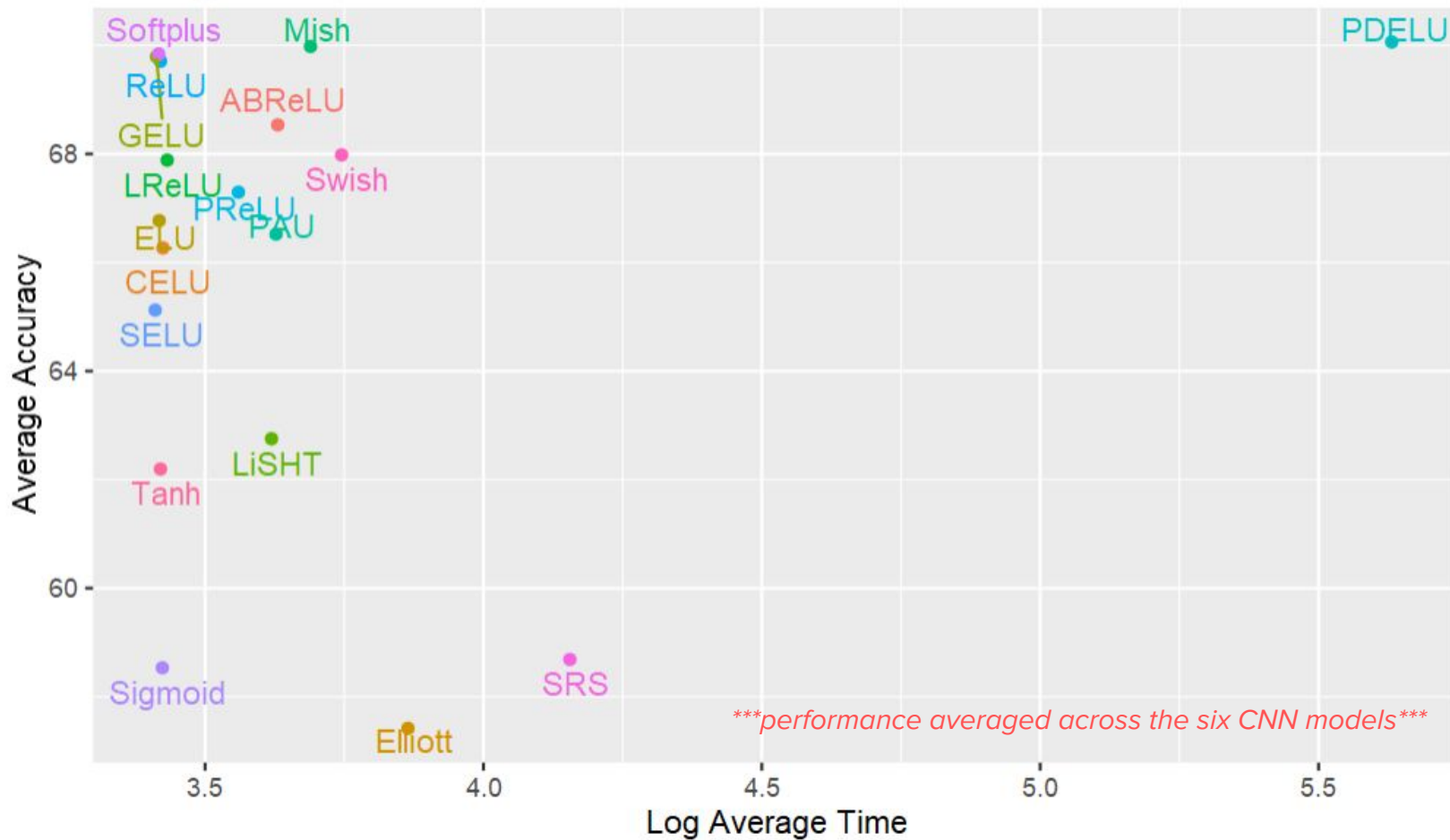


Top 20 Fastest Model-AF Performances

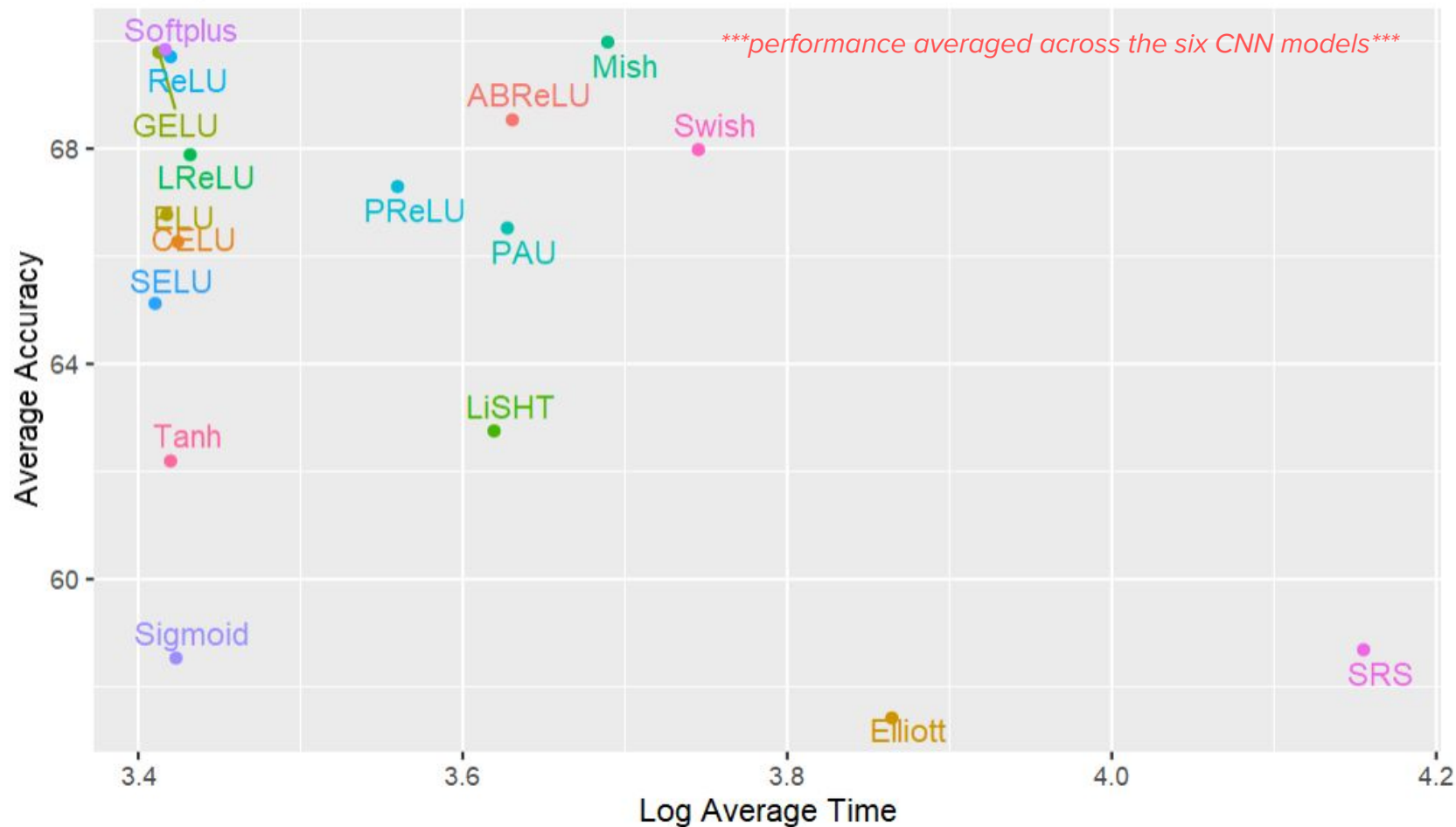


activations <chr>	AvgAccuracy <chr>	AvgTrainingTime <chr>
Sigmoid	58.533	30.67
Tanh	62.195	30.566
Elliott	57.418	47.678
ReLU	69.708	30.566
LReLU	67.887	30.938
PReLU	67.295	35.157
ELU	66.772	30.495
SELU	65.123	30.277
GELU	69.792	30.348
CELU	66.268	30.701
Softplus	69.843	30.465
Swish	67.982	42.319
ABReLU	68.535	37.735
LiSHT	62.753	37.312
SRS	58.687	63.765
Mish	69.98	40.019
PAU	66.523	37.619
PDELU	70.063	279.065

Average Performance of Each Activation Function

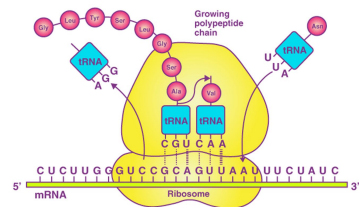


Average Performance of Each Activation Function



Translation

- German to English
- Model: Benchmark Seq2Seq - Long Short Term Memory (LSTM)
 - Autoencoder network
- 50 Epochs
- Learning rate 0.001
- Batch Size 256
- Adam optimizer used for training
- BLEU (Bilingual Evaluation Understudy) score
 - Measures similarity between machine-translation and high quality reference translations
 - Higher value = more accurate translation
 - Mean and standard deviation reported over 5 trials



not this kind

Speech Recognition

- Time-series signal data
- Deep Speech 2 framework (assemblyai)
- 100 hours of transcribed English audio
- Model: 2 CNN layers, 2 Recurrent Neural Network layers
- 29 classes
- Dropout factor 0.5, Learning rate 0.0005, batch size 10
- 10 Epochs
- Character Error Rate (CER) and Word Error Rate (WER)
 - Reported over 5 trials
 - Includes respective standard deviations

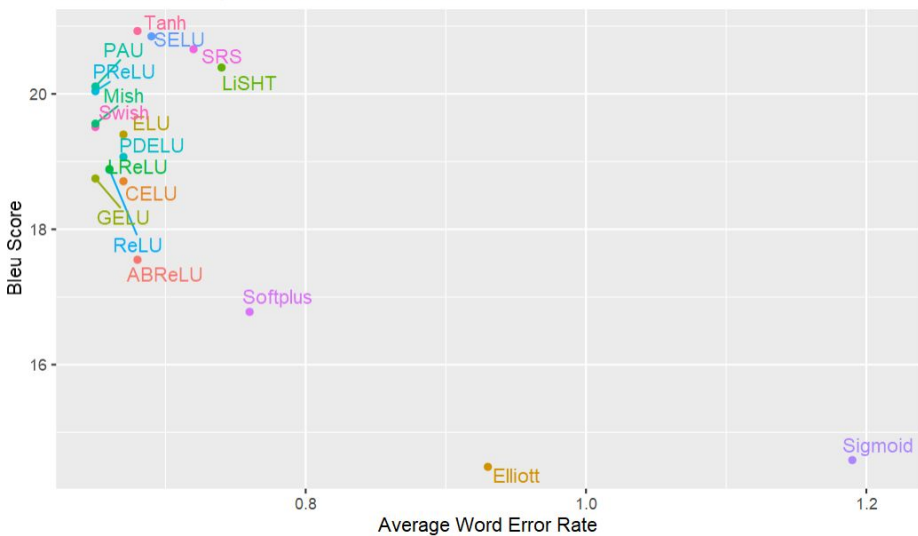


Table 11: Experimental results for German to English language translation and speech recognition tasks.

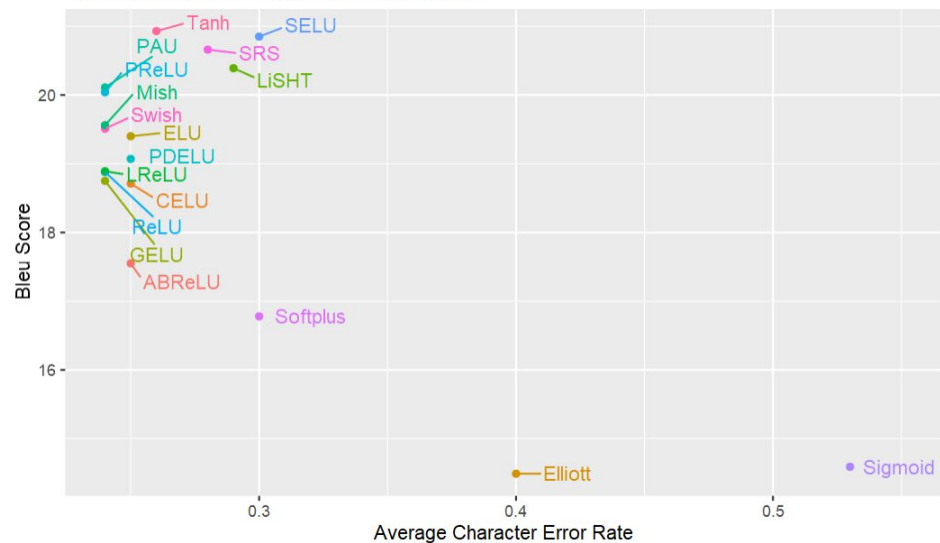
Activations	Language Translation	Speech Recognition	
	Bleu Score	Average CER	Average WER
Sigmoid	14.59 $\pm$ 0.47	0.53 $\pm$ 0.18	1.19 $\pm$ 0.39
Tanh	20.93 $\pm$ 0.91	0.26 $\pm$ 0	0.68 $\pm$ 0
Elliott [25]	14.49 $\pm$ 0.96	0.40 $\pm$ 0.01	0.93 $\pm$ 0.01
ReLU [8]	18.88 $\pm$ 0.86	0.24 $\pm$ 0.01	0.66 $\pm$ 0.01
LReLU [34]	18.89 $\pm$ 0.82	0.24 $\pm$ 0	0.66 $\pm$ 0.01
PReLU [35]	20.04 $\pm$ 0.98	0.24 $\pm$ 0	0.65 $\pm$ 0
ELU [27]	19.40 $\pm$ 1.33	0.25 $\pm$ 0	0.67 $\pm$ 0
SELU [52]	20.85 $\pm$ 0.64	0.26 $\pm$ 0	0.69 $\pm$ 0.01
GELU [101]	18.75 $\pm$ 1.83	0.24 $\pm$ 0	0.65 $\pm$ 0
CELU [53]	18.71 $\pm$ 0.55	0.25 $\pm$ 0	0.67 $\pm$ 0
Softplus [93]	16.78 $\pm$ 0.84	0.30 $\pm$ 0.01	0.76 $\pm$ 0.02
Swish [29]	19.51 $\pm$ 0.97	0.24 $\pm$ 0.01	0.65 $\pm$ 0.01
ABReLU [44]	17.55 $\pm$ 0.63	0.25 $\pm$ 0	0.68 $\pm$ 0
LiSHT [24]	20.39 $\pm$ 0.93	0.29 $\pm$ 0.01	0.74 $\pm$ 0.01
SRS [26]	20.66 $\pm$ 0.78	0.28 $\pm$ 0	0.72 $\pm$ 0
Mish [99]	19.56 $\pm$ 1.15	0.24 $\pm$ 0	0.65 $\pm$ 0
PAU [111]	20.11 $\pm$ 1.24	0.24 $\pm$ 0	0.65 $\pm$ 0.01
PDELU [59]	19.07 $\pm$ 0.95	0.25 $\pm$ 0	0.67 $\pm$ 0.01

From *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. Dubey, Singh, Chaudhuri

Translation vs Speech Rec Performance



Translation vs Speech Rec Performance



Summary

Summary (1 of 2)



- Attacking non zero-mean and zero-gradient problems leads to increased AF complexity
- ReLU
 - Attempts to solve issues of
 - Under-utilization of negative values
 - Limited nonlinearity
 - Unbounded output
 - Better in some applications than others
- ELU
 - Attempts to solve issues of
 - Under-utilization of negative values
 - Over-saturation of important features
 - Suffer due to non-smooth functions
- APL
 - Find best parameters to represent non-linearity needed for the given dataset
 - Difficulty in initializing training
 - Base function
 - Number of trainable parameters

Summary (2 of 2)



- Performance Analysis
 - Negative *and* positive values used to ensure near-zero mean
 - Overly complex models may lead to overfitting
 - Overly simple models may lead to poor convergence
 - Avoid Logistic Sigmoid and Tanh due to poor convergence
 - Parametric AFs demonstrate better convergence
 - Adapts to data more quickly - learns parameter from data
 - Accuracy vs training time
- PDELU generally very accurate but very slow
- Elliott and Sigmoid notably poor with language translation and speech recognition
- ReLU, ELU, CELU, Softplus good tradeoffs between accuracy and training time

Future Work

Analysis / Comparison of models

- Run times
- Robustness to different activation functions
- ***Across different data sets***

Experiments with

- Different batch sizes
- Learning rates
- computer RAM



References

Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark, Dubey et al (2022)

Smooth Function Approximation by Deep Neural Networks with General Activation Functions, Ohn & Kim (2019)

Advanced Learning Algorithms Coursera Yang

Questions
